

SAF-CNN: 面向嵌入式 FPGA 的卷积神经网络稀疏化加速框架

谢坤鹏^{1,4} 仪德智^{2,4} 刘义情^{2,4} 刘航^{1,4} 赫鑫宇^{2,4} 龚成³ 卢冶^{1,2,4,5}

¹(南开大学计算机学院 天津 300350)

²(南开大学网络空间安全学院 天津 300350)

³(南开大学软件学院 天津 300350)

⁴(天津市网络与数据安全重点实验室(南开大学) 天津 300350)

⁵(处理器芯片全国重点实验室(中国科学院计算技术研究所) 北京 100190)

(xkp@mail.nankai.edu.cn)

SAF-CNN: A Sparse Acceleration Framework of Convolutional Neural Network for Embedded FGAs

Xie Kunpeng^{1,4}, Yi Dezhi^{2,4}, Liu Yiqing^{2,4}, Liu Hang^{1,4}, He Xinyu^{2,4}, Gong Cheng³, and Lu Ye^{1,2,4,5}

¹(College of Computer Science, Nankai University, Tianjin 300350)

²(College of Cyber Science, Nankai University, Tianjin 300350)

³(College of Software, Nankai University, Tianjin 300350)

⁴(Tianjin Key Laboratory of Network and Data Security Technology(Nankai University), Tianjin 300350)

⁵(State Key Lab of Processors (Institute of Computing Technology, Chinese Academy of Sciences), Beijing 100190)

Abstract When deploying models on resource-constrained FPGAs, traditional convolutional neural network accelerators and inference frameworks often face challenges such as various device types, extremely limited resources, insufficient data bandwidth utilization, complex operator types that are difficult to match operators and schedule computing task reasonably. In this paper, a sparse acceleration framework of convolutional neural network (SAF-CNN) for embedded FPGA is proposed. Through the method of software and hardware co-design, SAF-CNN is jointly optimized from the two perspectives of hardware accelerator design and software inference framework. SAF-CNN first constructs parallel computing array and designs parallel encoding and decoding scheme to realize single-period multi-data transmission and effectively reduce communication costs. Secondly, a fine-grained structured block partitioning pruning algorithm is designed to obtain a sparse and regular weight matrix by cutting the input channel dimension within the block, so as to significantly reduce the computation scale and the resource utilization of DSP multiplier. Then, the input channel dimension dynamic expansion method and runtime scheduling strategy compatible with depth-separable convolution is proposed to realize flexible adaptation of input channel parameters and resource reuse of point-wise convolution and depth-wise convolution. Finally, the computational graph reconstruction method and hardware operator fusion are used to improve the hardware execution efficiency. The experiments use two

收稿日期: 2022-08-16; 修回日期: 2023-03-31

基金项目: 国家自然科学基金项目(62002175); 计算机体系结构国家重点实验室(中国科学院计算技术研究所)开放课题(CARCHB202016); 天津市企业优秀科技特派员项目(21YDTPJC00380); 中国民航大学信息安全测评中心开放基金项目(ISECCA-202102); CCF-华为胡杨林基金项目(CCF-HuaweiTC2022005)

This work was supported by the National Natural Science Foundation of China(62002175), the Open Project Fund of State Key Laboratory of Computer Architecture (Institute of Computing Technology, Chinese Academy of Sciences) (CARCHB202016), the Special Funding for Excellent Enterprise Technology Correspondent of Tianjin(21YDTPJC00380), the Open Project Foundation of Information Security Evaluation Center of Civil Aviation University of China (ISECCA-202102), and the CCF-Huawei Populus Grove Fund(CCF-HuaweiTC2022005).

通信作者: 卢冶 (luye@nankai.edu.cn)

resource-limited low-end FPGA heterogeneous platforms, Intel CycloneV and Xilinx ZU3EG. The results show that the SAF-CNN accelerator can achieve the computational performance of 76.3GOPS and 494.3GOPS respectively. Compared with multi-core CPU, SAF-CNN can achieve 3.5x and 2.2x performance improvement on the object detection model of SSD_MobileNetV1, and the model inference speed is up to 26.5fps.

Key words convolutional neural network; model compression; computational graph; accelerator design; inference framework

摘要 传统的卷积神经网络加速器及推理框架在资源约束的FPGA上部署模型时,往往面临设备种类繁多且资源极端受限、数据带宽利用不充分、算子操作类型复杂难以适配且调度不合理等诸多挑战.提出一种面向嵌入式FPGA的卷积神经网络稀疏化加速框架(sparse acceleration framework of convolutional neural network, SAF-CNN),通过软硬件协同设计的方法,从硬件加速器与软件推理框架2个角度进行联合优化.首先,SAF-CNN构建并行计算阵列,并且设计并行编解码方案,实现单周期多数据的传输,有效减少通信代价.其次,设计细粒度结构化块划分剪枝算法,于输入通道维度进行块内裁剪来获得稀疏且规则的权重矩阵,借此显著降低计算规模和DSP乘法器等资源占用.然后,提出一种兼容深度可分离卷积的输入通道维度动态拓展及运行时调度策略,实现输入通道参数灵活适配与逐通道卷积和逐点卷积的资源复用.最后,提出一种计算图重构及硬件算子融合优化方法,提升硬件执行效率.实验采用2种资源受限的低端FPGA异构平台Intel CycloneV与Xilinx ZU3EG,结果表明SAF-CNN加速器可分别实现76.3GOPS与494.3GOPS的计算性能.与多核CPU相比,SAF-CNN在进行SSD_MobileNetV1目标模型检测时,可实现3.5倍与2.2倍的性能提升,模型推理速度高达26.5fps.

关键词 卷积神经网络;模型压缩;计算图;加速器设计;推理框架

中图法分类号 TP391

随着边缘智能的发展,将深度神经网络部署在边缘低功耗设备来解决实际需求的需求愈加迫切^[1].卷积神经网络(convolutional neural network, CNN)是典型的深度神经网络,在目标检测、图像分类和语义分析等领域广泛应用^[2-4].然而,由于CNN模型计算量庞大通常在GB数量级^[5],而且CNN中主要计算类型与计算量集中在卷积操作^[6],其大量的乘累加计算与庞大的模型参数量对资源受限的边缘智能设备的模型部署带来了严峻挑战.

现场可编程门阵列(field programmable gate array, FPGA)凭借其灵活的设计模式和高效能优势,成为边缘智能领域部署深度学习各类模型及应用的理想平台^[7-9].然而,各类FPGA平台资源十分有限.因此,以往FPGA加速器设计时,针对深度神经网络模型计算量及参数量庞大的问题,主要通过2类方法来解决^[10]:一类是通过神经网络压缩(如模型量化和模型剪枝),来减少模型的参数并降低计算复杂度^[11-14];另一类是通过改进卷积的计算方式,获得轻量级的神经网络模型来降低计算量^[15].此外,在部署模型到硬件加速平台时,期望借助主流的神经网络推理框架如TensorFlow^[16],PyTorch^[17]等,以便缩短研发周期、实现快速部署^[18].但是,传统的深度神经网络加速器

设计及框架优化方法在面对资源受限的FPGA平台时,以CNN为例往往存在3个挑战:

1)神经网络模型所包含的算子操作类别愈发多样,FPGA平台需要为多种算子进行独立的硬件设计,资源消耗代价高昂.例如,MobileNet^[19]采用深度可分离卷积来代替传统卷积操作以降低参数数量和计算复杂度.深度可分离卷积可分为2种不同的计算方式,即逐通道卷积和逐点卷积^[19].逐通道卷积在计算输出特征图时只关联单一通道的输入特征图,而逐点卷积关联所有的输入特征图到每个输出特征图中.传统的FPGA加速器针对这2种不同的卷积操作分别设计不同的IP核进行计算加速^[20-21].然而,作为边缘设备的嵌入式FPGA资源往往受限,无法支撑分别为逐通道卷积和逐点卷积进行单独的硬件设计.

2)传统的CNN加速器设计方案对FPGA带宽资源利用不充分、数据交换通信代价高,尤其针对低位宽数据缺乏良好的处理机制.以往的研究工作通常采用切片方法将无法部署于FPGA的大块数据的计算分解为多个小块数据的计算,并通过优化数据交换机制来减少片上存储与片外存储之间的通信代价^[18,22],以便将模型部署于嵌入式FPGA.这些方法中,经过量化操作后其推理任务所使用的数据通常是低

位宽的定点数。然而,在片上与片外数据交换过程中,由于缺少对低位宽数据的有效组织,导致带宽资源利用不充分、并行读写效率低,从而成为制约高效计算的瓶颈。此外,直接封装多个低位宽数据为单个高位宽数据会降低 FPGA 端并行解码的执行效率。尽管采用独立分布的硬件资源可以满足并行解码的需求,但会造成碎片化的数据存储方式^[23-24],导致需要大量片上存储资源才能满足并行解码需求,资源代价高昂。

3) 边缘智能设备种类繁多且资源受限,嵌入式设备的资源相比于云中心设备是极端匮乏的,而传统的深度学习推理框架往往针对的是 GPU 等资源丰富的目标推理平台^[16],难以对边缘智能设备提供支持。由于目标设备与推理框架的特性不一致,导致模型部署时经常出现数据格式不匹配、算子描述不一致、计算流程定义各异等诸多无法规避的技术问题,从而严重降低边缘智能设备的推理性能。尽管推理框架如 TensorFlow Lite, Paddle Lite^[25]也可支持模型部署到嵌入式平台,但这些端侧框架在部署模型到 FPGA 平台时,其经过量化压缩后的神经网络模型的量化算子和非量化算子之间的数据类型描述存在差异,无法调度量化算子到 FPGA 上进行加速,进而造成模型计算中存留大量碎片化的计算图^[24]。这些大量的计算图将会导致 CPU 与 FPGA 这 2 种执行环境频繁切换,而每次执行环境的切换都将引入数据的拷贝和预处理操作,从而降低整体的模型推理性能。

为解决这 3 个挑战,提出一种面向嵌入式 FPGA 的卷积神经网络稀疏化加速框架(sparse acceleration framework of convolutional neural network, SAF-CNN),并将 SAF-CNN 的 FPGA 加速器设计和部署方法与推理框架优化方法实现为组件,在开源深度学习框架 Paddle Lite 中进行集成与优化,实现了 CNN 的快速部署。验证了 SAF-CNN 在 Intel CycloneV 和 Xilinx ZU3EG 这 2 种资源受限的 FPGA 平台上的推理性能。实验结果表明,SAF-CNN 在 2 种平台分别最大可实现 76.3GOPS 和 474.3GOPS 的计算性能。与多核 CPU 相比,针对目标检测模型推理,SAF-CNN 在这 2 种平台可提升推理速度最高达 3.5 倍与 2.2 倍。其中,SAF-CNN 在 ZU3EG 平台上可获得 26.5 fps 的优越性能。

本文的主要贡献包括 3 个方面:

1) 设计一种面向嵌入式 FPGA 的模型细粒度结构化块划分剪枝算法。根据各时钟周期所需计算的权重元素,将权重矩阵划分为不同的块矩阵。在块矩阵内进行输入通道维度上的结构化剪枝以获得稀疏

的且规则的权重,从而实现时钟周期粒度上的负载均衡,避免计算流水线执行被阻塞。

2) 提出一种兼容深度可分离卷积的输入通道维度动态拓展及运行时调度策略。根据硬件资源可支持的计算阵列处理单元数量,动态调整逐通道卷积权重的输入通道数量,并在运行时调度逐通道卷积与逐点卷积到相同的计算、存储和数据传输模块,从而实现高效的资源复用。

3) 提出一种针对量化 CNN 模型的硬件算子类型感知的计算图重构方法。在量化模型的计算图中,依据硬件平台的算子库快速搜索数据类型存在差异的量化算子。通过引入量化节点,增加硬件子图中的算子数量,进而减少 CPU 与 FPGA 执行环境的切换次数,降低不同平台数据拷贝和转换的代价。

1 相关工作

面向嵌入式 FPGA 的稀疏化 CNN 加速框架研究主要涉及 CNN 模型压缩、CNN 加速器设计,以及推理框架联合优化 3 个方面,其中模型压缩方法主要聚焦于 CNN 模型量化与 CNN 模型剪枝。

在 CNN 模型量化算法方面,其方法可分为低比特量化^[11]、线性量化^[26]以及非线性量化^[27]三类。低比特量化是通过极低的比特位数的量化来提高计算效率,模型权重通常只有 1 个或 2 个比特位,如二值化神经网络 BNN^[11]和三值化神经网络 TWN^[12]。尽管二值化和三值化的神经网络可以实现更高的计算效率,但其精度下降严重,难以实际应用。多比特的线性量化和非线性量化 2 类方法已经被成功用于各类神经网络模型和推理框架^[14]。线性量化首先寻找缩放因子,并将其数据缩放到目标范围,然后采用舍入操作实现阈值化处理。与线性量化相比,非线性量化方法更适合处理神经网络中符合钟形分布和长尾数据分布的权值与激活值。对于 SAF-CNN,本文权衡模型精度与方法通用性,采用线性方法实现模型量化。

在 CNN 模型剪枝方面,通过裁剪 CNN 中冗余的参数,可有效减少 CNN 的存储参数并降低计算规模^[28]。剪枝方法可分为非结构化剪枝和结构化剪枝^[29]。非结构化剪枝将 CNN 中每个独立的连接看作为可被裁剪的对象,能够实现理想的剪枝率。但是,其不规则的稀疏性难以适配通用硬件体系结构,通常需要进行硬件定制化设计^[30]。例如, Song 等人^[31]设计了基于高带宽存储器的稀疏矩阵向量乘的专用加速器,实现了以内存为中心的处理引擎,可支持稀疏

的矩阵向量乘,但其严重依赖高带宽大容量的片上存储器,难以应用到边缘计算平台.结构化剪枝则按照不同的粒度,将CNN权值划分为大小不同的结构进行相应裁剪,可在通用硬件体系结构下实现加速,但是剪枝率往往偏低^[32].通道剪枝^[33]和卷积核剪枝^[29]是2类常用的结构化剪枝方法.通道剪枝直接剪去输出通道的所有卷积核,生成的稀疏化模型可直接部署于现有神经网络加速器而无需修改硬件.卷积核剪枝方法则是剪去固定输入通道的卷积核,其剪枝粒度相比通道剪枝的方法较细.然而,通道剪枝和卷积核剪枝直接修改了模型的结构,其剪枝粒度仍旧难以实现较高的模型稀疏率.细粒度剪枝通过构建专用的剪枝算法实现较高的稀疏率,以此可达到比肩非结构剪枝的效果^[34].基于硬件感知的细粒度剪枝方法,例如块内剪枝^[35],既可以缓解硬件效率低的问题,也可以提升模型的稀疏率.对于SAF-CNN,本文权衡模型精度与硬件设计及其执行效率后,设计结构化的细粒度剪枝方法,获得稀疏化的CNN模型.

在嵌入式FPGA加速器设计方面,由于CNN卷积层的乘累加计算占据了模型中90%左右的计算量^[6],因此基于FPGA的CNN加速器研究往往从计算和通信2方面对计算密集的卷积层进行优化^[18].Wei等人^[22]提出脉动阵列的架构实现卷积层计算,能够拓展到大规模计算平台加速卷积计算,大幅度降低推理延迟.Zhang等人^[36]采用多种嵌套循环优化策略加速卷积层处理,如循环流水线、循环展开和循环平铺.卢冶等人^[37]从数据访问独立性的角度提出通道并行计算的方案,引入循环切割因子并在通道维度展开循环,利用数据流优化和缓存优化技术加速CNN的卷积计算.Lu等人^[38]利用Winograd算法降低卷积层计算复杂度,提高卷积层的处理速度.然而,文献[36–38]所述工作中的普通卷积计算加速器主要集中在输入输出通道维度来并行加速,对于深度可分离卷积的逐通道卷积不能进行计算,导致普通卷积加速器难以适用于深度可分离卷积.对于设计支持深度可分离卷积的加速器,Wu等人^[39]和Ding等人^[21]相继提出逐通道卷积和逐点卷积的融合机制,实现跨层的融合优化,减少中间数据的通信带宽需求,提升深度可分离卷积的处理性能.然而,这类融合机制的本质仍基于2个独立IP核,而且对模型的结构具有严格的约束,只适用于具有1×1卷积核的逐点卷积.Zhang等人^[40]在处理深度可分离卷积时,对逐通道卷积和逐点卷积也采用2个IP核轮流工作.这些工作都未针对逐通道卷积和逐点卷积设计可复

用的计算引擎,导致较高的硬件资源代价.Yan等人^[41]针对深度可分离卷积提出可配置加法树的方案,实现了逐通道卷积和逐点卷积共享计算引擎,但因缺少共享数据传输模块,导致资源消耗偏高.对于加速器通信优化,ShiDianNao^[42]将所有的模型参数存储于片上SRAM中,消除对DRAM的访问,但该设计需要较高存储资源,并不适用于大规模的CNN模型.FitNN^[43]提出了一种跨层数据流策略来减少特征图的片外数据传输,基于资源受限的边缘设备实现低推理延迟,然而该策略只支持特定的模型结构,即只由逐通道卷积层、逐点卷积层和池化层共同构成的模型.此外,Cong等人^[6]分析了如何利用高位宽模式传输数据,将多个32b的字打包为512b的字,实现单周期传输多数据,最大化利用带宽资源.但是,该方法没有考虑片上存储资源的分配方式,导致并行编码和解码消耗大量存储资源.因此,本文在设计SAF-CNN时,采用资源复用方法,映射逐通道卷积与逐点卷积到可复用的硬件资源,然后设计通道维度上的并行编解码方法实现单周期传输多数据,在有限硬件资源上实现计算与数据传输效率最优.

在深度学习推理框架设计方面,通常会将神经网络的计算流程以计算图的方式呈现^[44],在实际应用中计算图被定义为有向图,其中节点代表相应的计算操作,边代表数据的流入与流出.深度学习框架中的计算图主要分为静态计算图和动态计算图^[16].在模型开发时采用动态图编程以获得更好的编程体验、更易用的接口、更友好的调试交互机制^[17].在模型训练或者推理部署时,通过添加装饰器将动态图代码转写为静态图代码,并在底层自动使用静态图执行器运行,获得更好的模型运行性能^[25].主流深度学习训练框架PyTorch^[17]采用动态计算图,TensorFlow^[16]采用静态计算图,而PaddlePaddle^[25]等新兴框架则采用动静融合的方案.此外,推理框架的优化方法主要聚焦于:算子融合、内存优化、子图检测和生成、硬件平台相关优化、冗余算子删除、精度转换等.对于FPGA加速器与深度学习框架集成实现联合优化方面,Li等人^[45]集成卷积计算的前向传播算子和反向传播算子到TensorFlow的算子库中,加速CNN模型的训练和推理.FADES^[46]提出一种针对TensorFlow Lite的稀疏矩阵和密集矩阵的硬件处理架构,支持CPU和FPGA异构加速器,并能从结构化剪枝中获益,进一步提升计算性能.Zhang等人^[18]提出Caffeine,通过集成卷积计算和全连接计算的算子到Caffe框架的算子库中,来加速CNN模型的推理.然而,FADES

和 Caffeine 并未涉及计算图的优化,在推理过程中需进行数据转换,执行环境的频繁切换造成额外的时间代价. TVM^[44] 和 Paddle Lite 的推理后端支持多种定制硬件平台,如 NPU, ARM, x86, MLU 等神经网络加速器,但是缺少对嵌入式 FPGA 的平台的支持. 因此,本文在设计 SAF-CNN 时也充分探索基于计算图的优化方法,既要发挥加速器计算性能高的优势,又要充分利用深度学习框架的可拓展性和通用性.

2 SAF-CNN 稀疏加速器设计

本节首先介绍 SAF-CNN 稀疏化加速器的总体架构,然后着重就数据流设计、稀疏化计算引擎、数据存储与传输、算子资源复用 4 个方面,展开加速器设计思路和优化方法的具体论述,并阐明 SAF-CNN 稀疏加速器的优势.

2.1 SAF-CNN 稀疏加速器架构

本文 SAF-CNN 所设计的 FPGA 架构如图 1 所示,包含数据引擎、片上缓存单元、指令单元、计算引擎及调度模块 5 个重要组成部分.

1) 数据引擎. 负责在片上存储和片外存储之间进行数据传输. FPGA 加速器在运行过程中读取片外存储器中的输入特征图和模型权重参数,以及写出输出特征图,都需通过该模块进行数据传输. 此外,SAF-CNN 采用高位宽的数据传输模式,单个周期传输多个字节,借此充分利用带宽资源.

2) 片上缓存单元. 片上存储器负责缓存输入特征图、模型权重参数以及输出特征图的数据,为计算单元提供并行访问的高带宽支撑. SAF-CNN 采用了双缓冲的存储结构来实现计算与数据传输并行执行.

3) 指令单元. 在 SAF-CNN 框架中,卷积层的配置被转化为固定格式的指令. FPGA 加速器启动时,通过指令单元获取卷积层的具体配置参数,并对硬件资源进行配置. 在加速器运行过程中,其他模块的执行都依赖指令的具体配置.

4) 计算引擎. 对于从片上存储器读取的输入特征图和稀疏化模型权重参数,SAF-CNN 的计算引擎通过乘法器和加法器来执行卷积操作,是卷积(conv)模块的核心. 此外,本文在 SAF-CNN 中设计了稀疏化的卷积计算模块,以此利用权重稀疏性来加速卷积层处理.

5) 调度模块. 对于深度可分离卷积的逐通道卷积和逐点卷积,SAF-CNN 设计了可复用的计算引擎和数据传输模块,调度模块根据具体计算类型来决定数据传输模块所要读写的数据地址,以及计算模块的执行流程.

2.2 SAF-CNN 数据流设计

由于片上资源受限,SAF-CNN 采用数据分块策略来完成卷积层的计算. SAF-CNN 的卷积计算数据流处理包含了 3 个模块,分别为加载(load)、卷积(conv)和存储(store). 在卷积操作完整的计算流程中,3 个模块对应的子任务不断迭代执行. 例如,在每次

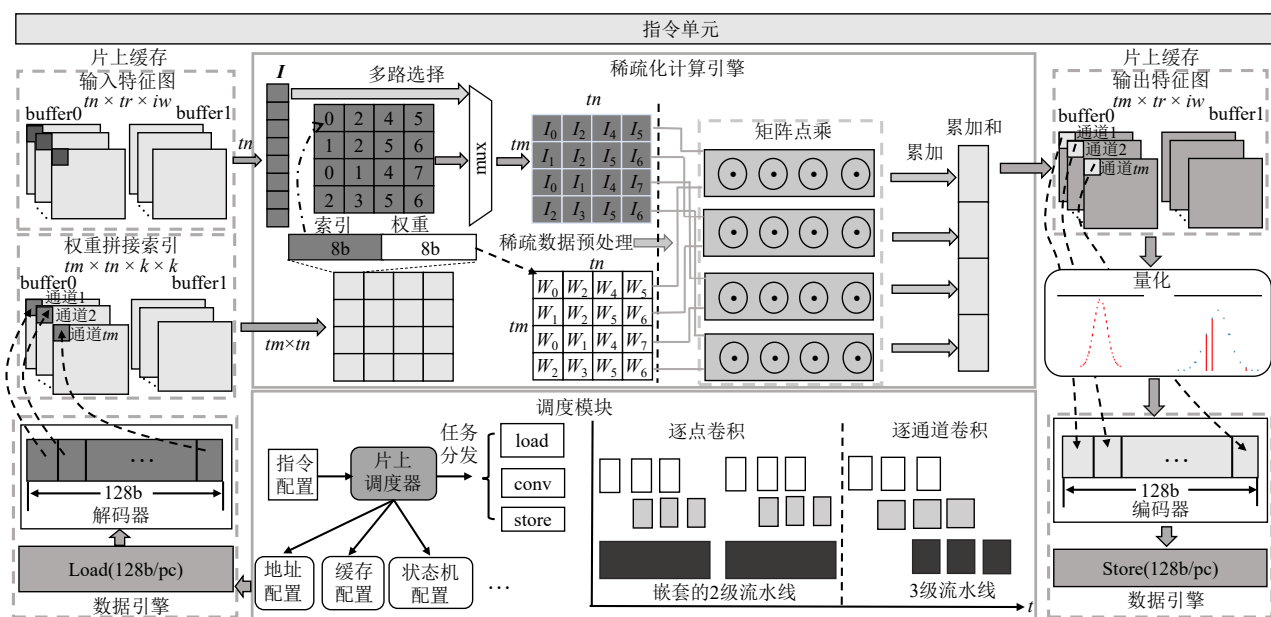


Fig. 1 Overall architecture of FPGA sparse accelerator

图 1 FPGA 稀疏化加速器整体架构

迭代中,可先累加输入特征图与卷积核的卷积结果,得到输出特征图.具体来讲,首先 load 模块加载必要的输入特征图数据和权重数据到片上存储器中.然后,conv 模块采用流水并行的稀疏化计算引擎对片上存储器中的数据执行卷积计算.最后,在完成输出特征图的计算后,store 模块写回输出特征图到片外大容量存储器中,而后进入到下一个输出特征图分块的计算过程.

数据流的相关处理如图 2 所示,其中 oc , oh , ow

分别表示输出特征图的通道数、行数和列数, ic 表示输入特征图的通道数, $kernel$ 表示卷积核的大小.受片上资源约束,SAF-CNN 每次迭代只计算 tm 个通道、 tr 行和所有列的输出数据.考虑到不同卷积层具有各自不同的行数和列数,SAF-CNN 分配固定大小的片上缓存, tr 则根据列的大小进行相应变化,以此保证所有的列可被完整计算.通过这种设计,输入特征图和输出特征图的数据传输方能利用 FPGA 的猝发式^[14]机制,从而充分利用带宽资源.

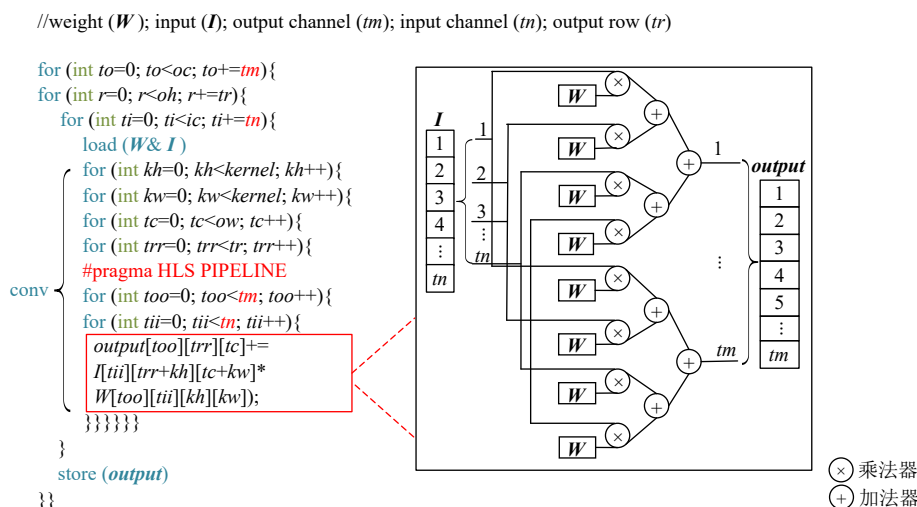


Fig. 2 Computing dataflow of convolution

图 2 卷积计算数据流

在 conv 模块执行卷积计算时,计算引擎沿着输入通道和输出通道 2 个维度同时并行.输入通道并行因子和输出通道并行因子分别是 tm 和 tm .计算引擎使用 tm 个累加树,通过共享 tm 个输入数据来计算得出 tm 个输出数据.每个累加树包含了 tm 个乘法单元和 $tm-1$ 个加法单元,负责计算 1 个输出元素.通过并行 tm 个累加树,计算引擎可同时计算并得出 tm 个输出元素.此外,由于 SAF-CNN 计算引擎采用深度流水线来提升吞吐率,每个周期都能计算出 tm 个输出元素.通过设计并行和流水的硬件架构,SAF-CNN 中的 conv 模块可以最大化资源利用率,并提升输入特征图分块的吞吐率.

为减少 conv 模块的数据准备等待时间,SAF-CNN 将数据传输与计算进行并行,通过预先准备数据,从而掩盖数据传输的时间代价.对输入特征图、权重和输出特征图,SAF-CNN 采用双缓冲存储结构来实现一边传输数据一边为 conv 模块提供数据.

2.3 稀疏化计算引擎

本节将详述 SAF-CNN 稀疏化计算引擎设计中

剪枝方法如何择取、剪枝规则如何设定、剪枝后的数据如何保存,以及剪枝后如何进行计算.

1)对于剪枝方法的择取,考虑到 conv 模块中的计算引擎负责执行矩阵向量乘操作,共使用 $tm \times tm$ 个乘法器.具体来讲,如图 2 所示,对于一个权重张量 W (tm 行、 tm 列)和一个输入向量 I (长度为 tm),通过计算可得长度为 tm 的输出向量.通过剪去张量 W 中冗余的权重参数,可以有效地降低乘法器的使用数量.考虑到流水线过程中计算引擎的硬件资源不断被重复使用,非结构化的稀疏方式会造成不规则的存储访问和不均衡的计算负载,导致流水线被阻塞,而结构化的稀疏方式可避免此现象.因此,SAF-CNN 稀疏化加速器采用结构化剪枝来保证每次参与计算的权重矩阵具有相同的稀疏率.

2)对于剪枝规则的设定,考虑到张量 W 的大小是 $oc \times ic$,conv 模块的计算引擎每次计算的块大小是 $tm \times tm$,因此构建基于块的剪枝方法,可以保证每次计算的块具有相同的稀疏度.SAF-CNN 的卷积层权重在输入通道维度所进行的块内结构化剪枝方法的具

体细节如图3所示. 其中, 每行元素的个数是 ic , 可划分为 ic/t_n 个块, 每个块包含 t_n 个元素. 在每一块内, 可设置固定的稀疏率, 绝对值更小的元素则被置 0. 此外, 所有的行采用相同的稀疏率. 针对块内的每一行, t_n 个权重元素和 t_n 个输入数据乘累加得到输出数据. 剪掉 t_n 个权重数据中绝对值更小的项等价于舍弃 t_n 个累加项中更小的项, 这种方式可以有效地裁剪掉对输出数据影响更小的权重.

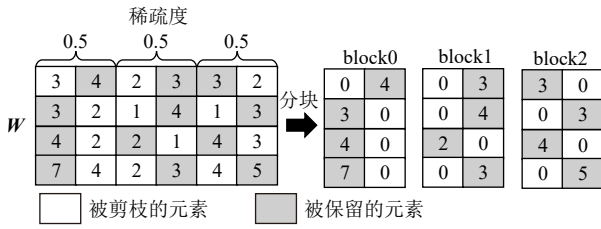


Fig. 3 Block-based structured pruning method along input channel

图3 输入通道块内结构化剪枝方法

3) 对于剪枝后的模型数据存储, 为避免 0 元素参与计算, SAF-CNN 对权重进行压缩处理, 直接舍去 0 元素, 例如图3中的 block0 可以压缩为 4 行 1 列的矩阵. 同时, SAF-CNN 采用列索引方式保存非零元素的位置信息. 与 CSR/COO^[47] 等稀疏化数据存储方式不同, 块内结构化剪枝方法只需要保留列维度的位置信息. 考虑到量化后的权重数据为低位宽的定点数, 索引数据使用的位宽定义为 $\lceil \lg(t_n) \rceil$. 因此, 权重的非零元素和对应的索引数据可以拼接在一起. 由于推理过程中, 权重数据是固定不变的, 因此压缩和拼接均在离线条件下完成.

4) 对于剪枝后的模型如何计算, SAF-CNN 为充分利用权重矩阵的稀疏性, 构建了稀疏化的计算引擎, 负责稀疏化的矩阵向量乘操作. 稀疏化计算引擎的具体内容如图1所示, 包含 2 个模块, 采样模块和乘累加模块. 首先, 片上的权重数据分离出非零权重矩阵和索引矩阵. 然后, 索引矩阵和输入向量进入到采样模块, 数据多路选择器根据索引矩阵从输入向量中挑选指定的数据得到输入矩阵, 该矩阵每一行的元素是输入向量的子集. 接下来, 具有相同元素个数的非零权重矩阵和采样模块生成的输入矩阵进行相乘, 得到中间矩阵. 最后, 累加中间矩阵的每一行, 即可得到目标输出向量. 由此, SAF-CNN 计算引擎设计充分利用了权重矩阵每一行具有相同的稀疏度的特点, 即包含相同个数的非零元素, 并使用相同个数的乘法器和累加器进行计算. 由于块内结构化的剪

枝方法可使不同的分块具有相同的稀疏规则, 因此, 稀疏化计算引擎能够高效地处理所有分块, 并且对不同分块具有相同的处理时间, 可保证卷积计算流水线不会阻塞, 即 SAF-CNN 所设计的稀疏化计算引擎中的乘法器和累加器都将处于忙碌状态, FPGA 计算资源被高效利用.

2.4 数据存储与传输

数据存储方式是影响加速器性能的重要因素之一^[14]. 针对片上存储, SAF-CNN 稀疏化加速器为避免硬件资源访问端口数目限制而导致的数据访问冲突, 设计了满足单周期多数据并行访问需求的存储方案. 对于片外存储, SAF-CNN 利用数据重组织机制保证数据分布的连续性, 来提升数据通信效率.

对于片上存储, 将输入特征图、权重和输出特征图数据分布在不同的存储块(bank)上, 可符合计算引擎并行的数据访问要求. 每个 bank 具有 1 个数据读写端口, 每个周期内可以执行 1 个数据的读写操作. 计算引擎采用流水并行的硬件架构, 每个时钟周期读取 t_n 个输入数据和 $t_m \times t_n$ 个权重数据, 写入 t_m 个输出数据. 此外, 每个时钟周期读写的数据位于不同的通道. SAF-CNN 加速器设计利用高层次综合 HLS 工具提供的 pragma 优化指令, 针对输入特征图、权重和输出特征图数据, 在通道维度上进行分割, 使不同的通道数据可被存储在不同的 bank 上, 从而满足数据并行访问需求.

对于片外存储, 由于数据流的分块方式, 加速器对多维张量的数据访问具有局部性^[14], 导致 load 模块和 store 模块每次所访问的数据在片外存储器上是碎片化分布的. 访问碎片化的数据将严重降低数据传输性能. 为了最大化提升数据传输性能, SAF-CNN 利用重组织操作将片外存储中的碎片化数据进行整理, 从而形成连续的数据分布. 输入特征图张量各个维度的大小是 (ic, ih, iw) , ic, ih, iw 分别是输入通道、输入行和输入列的大小. 经过重组织后的输入特征图分布为 $(\lceil ic/t_n \rceil, ih, iw, t_n)$. 模块 load 每次加载的数据大小是 $t_r \times iw \times t_n$, 这些数据在片外存储器上是连续分布的. 此外, 输出特征图采用相同的重组织方式. 未经处理的权重分布为 $(oc, ic, k \times k)$. 其中, k 表示卷积核的大小, 则经过重排后的权值分布为 $(tb, k \times k, t_m, t_n)$, 进一步将稀疏化的权重从 t_n 压缩为 dn , 得到 $(tb, k \times k, t_m, dn)$. 其中, tb 是指权重拷贝的次数, $tb = \lceil oc/t_m \rceil \times \lceil ic/t_n \rceil$, 每次拷贝的块大小则为 $k \times k \times t_m \times dn$. 为了降低数据重组带来的额外代价, SAF-CNN 对权重进行离线压缩和重组, 如算法 1 所示.

算法 1. 卷积层权重数据压缩重组算法.

输入: 原始权重张量 W , 原始掩码张量 M , 输出通道数 oc , 输入通道 ic , 卷积核大小 $k \times k$, 输出通道并行因子 tm , 输入通道并行因子 tn , 剪枝后的输入通道并行因子 dn ;

输出: 重组后的权重张量 W' .

- ① $booc \leftarrow \lceil oc/tm \rceil$;
- ② $bioc \leftarrow \lceil ic/tn \rceil$;
- ③ for all $bo < booc$ do
- ④ for all $bi < bioc$ do
- ⑤ for all $ti < dn$ do
- ⑥ for all $tk < k \times k$ do
- ⑦ for all $m < tm$ do
- ⑧ computing $mask_index$ using
 bo, bi, ti, tk, m ;
- ⑨ $mask = M[mask_index]$;
- ⑩ select $weight$ from W by $mask$;
- ⑪ computing $weight_index$ using
 bo, bi, ti, tk, m ;
- ⑫ $W'[weight_index] = weight$;
- ⑬ $W'[weight_index+1] = mask$;
- ⑭ endfor
- ⑮ endfor
- ⑯ endfor
- ⑰ endfor
- ⑱ endfor

算法 1 中, 行⑧⑨用来计算目标掩码数据在原始掩码矩阵的索引, 并赋值给 $mask$, 行⑩利用 $mask$ 从原始权重张量中选择数据 $weight$, 行⑫⑬将 $weight$ 和 $mask$ 拼接到一起并放置在目标权重张量中. 此外, 加速器将输出数据写回片外存储器时, 直接对输出数据进行重组. 在执行接下来的卷积层时, 输入特征图已经被处理, 不需要额外的重组过程. 通过这种设计, SAF-CNN 只需要对 CNN 模型的第 1 层和最后 1 层卷积层的数据进行重组, 有效地降低数据重组引入的时间代价.

片外存储器的数据经过重组以后, 每次传输输入特征图数据、权重数据和输出特征数据都将符合连续分布. 由此, 可采用猝发式传输方法高效地传输这些数据. 为了进一步利用带宽资源, 本文采用高位宽的传输模式, 在单个周期内传输多个低位宽的数据. 通过使用 128b 的传输位宽, 单个周期可以传输 16 个 8b 的数据. 经过重组后, 高位宽传输的批量数据处于不同的通道. 在片上存储器中, 不同通道的

数据位于不同的 bank. 因此, 单个周期读取的多个数据可以并行地写到多个 bank, 或者从多个 bank 读数据写回到片外存储器中. 通过结合重组方法和高位宽数据传输模式, SAF-CNN 可以充分利用硬件平台提供的带宽资源.

2.5 资源复用设计

SAF-CNN 针对特殊卷积操作如深度可分离卷积进行了资源复用设计. 深度可分离卷积是一类特殊的卷积运算, 其有逐通道卷积和逐点卷积 2 种卷积方式. 逐通道卷积在计算输出特征图时, 只计算单一通道的输入特征图; 而逐点卷积则计算所有通道的输入特征图并累加到输出特征图中. 其中, 逐点卷积使用的卷积核大小是 1×1 . 考虑到加速器的可拓展性, SAF-CNN 采用通用卷积的配置模式, 并灵活支持卷积核尺寸. 为节省资源, SAF-CNN 通过资源复用设计, 将逐通道卷积和逐点卷积调度到同一数据传输模块和同一计算引擎, 2 种卷积计算的主要区别在于是否沿着输入通道进行累加, 其计算方式取决于调度策略.

在 SAF-CNN 中, conv 模块的核心即计算引擎, 它采用累加树规则对 tn 或者被压缩后的 dn 个 (如 2.4 节所述) 输入通道的乘积进行累加. 然而, 逐通道卷积并未在输入通道累加, 而是通过计算单一通道的输入特征图得到输出特征图. 因此, SAF-CNN 为了将逐通道卷积映射到计算引擎上, 采用了填充策略对逐通道卷积的权重进行预处理, 如图 4 所示. 其中, 在未采用剪枝策略时, tm 个输出通道的权重拓展为 $tm \times tn$ 个通道, 这种填充策略要求 $tm=tn$. 在填充后, 输出通道和输入通道序号相同的卷积核包含有效数据, 其他通道卷积核为 0 元素. 如图 4(a) 所示, 原始的逐通道卷积第 i 个输出通道的数据, 需要被放置在第 i 个输入通道上, 以此保证计算引擎在累加 tm 个输入

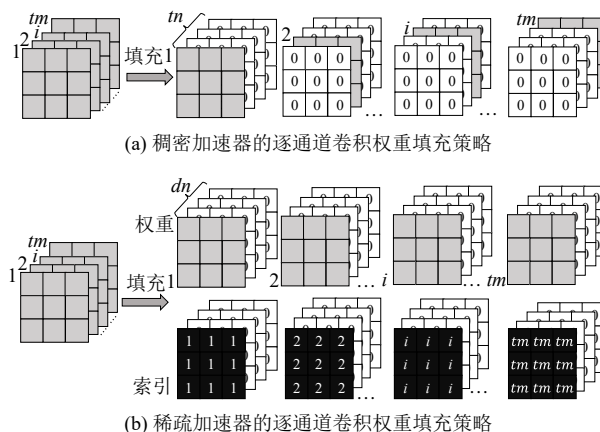


Fig. 4 Weight expanding strategy of depth-wise convolution

图 4 逐通道卷积权重填充策略

通道的数据时,能够正确计算出逐通道卷积的输出特征图.采用剪枝策略后,权重被分为2个部分,即非零权重和索引信息.SAF-CNN利用索引信息将逐通道卷积的计算映射到稀疏化的计算引擎,如图4(b)所示.其中, tm 个输出通道的权重拓展为 $tm \times dn$ 个通道,逐通道卷积的第 i 个输出通道数据可被放置在第1个输入通道,其他输入通道数据置0.同样,第 i 个输出通道的索引矩阵只在第1个输入通道上包含有效数据,具体的数据被设置为 i .这种填充方式可以保证稀疏化的计算引擎既可以执行稀疏的逐点卷积也可以执行逐通道卷积.此外,针对经过剪枝后的SAF-CNN稀疏加速器的逐通道卷积权重填充策略,其 tm 和 dn 可以灵活取值便于计算引擎的设计.

除了共享复用计算引擎外,逐通道卷积和逐点卷积还可共享复用数据传输模块.在每次分块计算过程中,逐通道卷积和逐点卷积需要传输的数据块具有相同的大小和不同的地址偏置.因此,对于2种不同类型的卷积计算,通过调度策略可以映射到相同的数据传输模块和计算模块,达到复用效果.调度策略的编程实现示例如图5所示.对于逐通道卷积,load, conv, store这3个模块轮流执行.如2.4节所述,由于片上存储采用双缓冲的存储结构,因此可对3个模块进行流水线执行.对于逐点卷积,需要遍历其输入通道,并将结果累加到输出特征图.因此,load模块和conv模块所形成的计算流水线需多次执行.当输入通道计算完毕,store模块负责写回输出特征图.

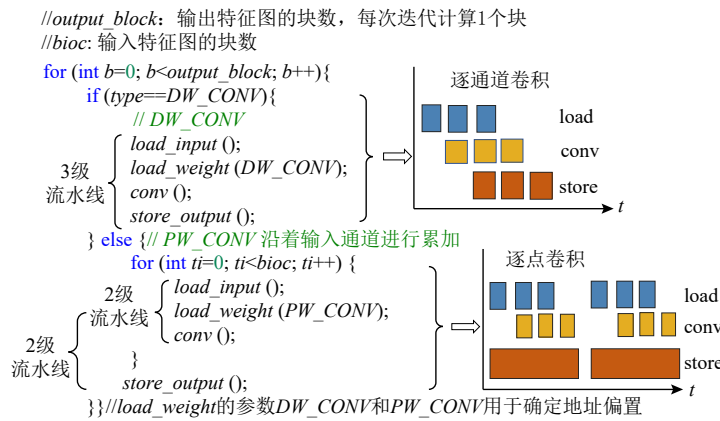


Fig. 5 Scheduling strategy of convolution layer

图5 卷积层调度策略

3 SAF-CNN 推理框架实现

为便于快速部署各类 CNN 模型到嵌入式 FPGA 平台,本文设计实现了 SAF-CNN 推理框架.本节将论述 SAF-CNN 推理框架的具体设计及优化方法,重点聚焦于框架推理流程、算子融合方法、模型重构及硬件子图融合方法.SAF-CNN 通过把加速器的设计思想与推理框架结合并进行联合优化,弥补了互相割裂的稀疏化加速器与推理框架,从而支持快速部署各类 CNN 模型.

3.1 SAF-CNN 框架推理流程

如图6所示,利用 SAF-CNN 推理框架来部署稀疏化 CNN 模型到嵌入式 FPGA 的主要环节有3个方面:

1) 首先利用训练框架(实验中基于 PaddlePaddle)训练得到精度理想的目标模型.其次,通过训练框架所集成的 QAT^[25]等量化方法对模型权重和激活值进

行量化,并嵌入2.3节所设计的结构化剪枝方法到训练框架中进行模型剪枝,SAF-CNN通过掩码方法^[28]将冗余的权重参数过滤.然后,通过模型计算图重构的方法来分析模型的计算图结构,依据目标硬件平台的算子库搜索数据类型不匹配的量化算子如卷积计算、全连接计算等,在该量化算子之前插入符合硬件数据类型的量化节点,并微调 CNN 模型保证精度.

2) 利用训练框架导出静态图模型后,针对其计算图进行优化,主要包含算子融合、量化节点卸载到 FPGA、生成硬件计算子图、模型权重预处理.其中,算子融合优化主要针对 Conv 层、ReLU 层和 BN 层进行融合,并重新计算权重参数、偏置和量化参数.量化节点卸载到 FPGA 可与卷积计算融合.在计算图中,目标硬件支持的具有数据依赖关系的算子所构成的集合称作子图^[25],将子图中计算节点参数配置转化为硬件支持的参数配置.借此,目标硬件可一次执行多个计算任务,减少执行环境的切换次数.此

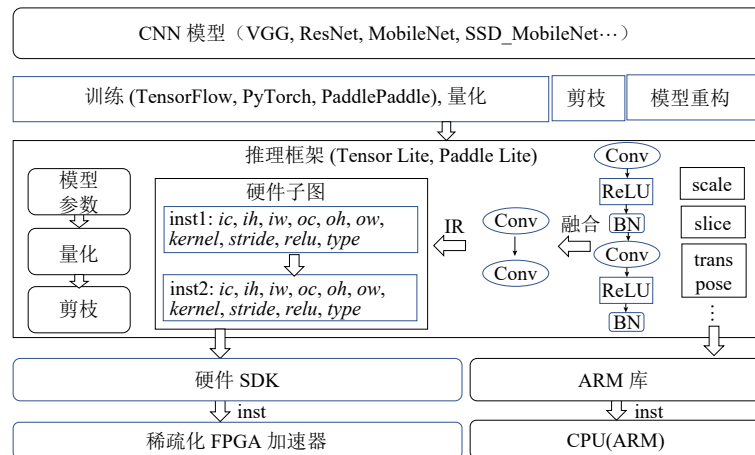


Fig. 6 Integrating sparse accelerator and deep learning framework

图6 集成稀疏化加速器和深度学习框架

外,训练框架导出的模型参数类型通常是浮点数,利用量化参数和剪枝所采用的掩码方法可对模型参数进行量化和剪枝预处理操作。

3) 针对 CPU-FPGA 异构架构,SAF-CNN 采用混合调度策略进行推理,由逐通道卷积和逐点卷积算子构成的子图,通过硬件运行时接口 SDK 被调度到加速器平台执行,其他轻量级计算任务保留在 CPU 端执行.SDK 对 CNN 模型的第 1 层的输入特征图和最后 1 层的输出特征图进行数据重组,硬件子图遍历,并启动 FPGA 进行计算。

3.2 算子融合

算子融合是计算图优化的常用方法^[44],通过将多个算子融合为单个算子,可避免中间数据的存储和传输带来的时间消耗,从而降低推理延迟。卷积层、批归一化(batch normalization, BN)层和激活层是 CNN 模型中普遍使用的算子,且这些层的计算可被融合为 1 层计算。在引入量化操作后,卷积层和 BN 层的权重参数与量化参数可以进行融合。经过算子融合后,在第 i 个输出通道的输出特征图计算定义为

$$output_i = is \times ws_i \times output_i + bias_i, \quad (1)$$

其中 $output$ 是量化后的输入特征图与权重进行卷积计算所得到的定点数张量,经过输入缩放因子 is 和权重缩放因子 ws 的缩放操作后,累加偏置参数 $bias$ 得到正确的浮点数输出特征图 $output'$ 。此外, BN 层的缩放参数已被融合到 ws ,偏置参数已被融合到 $bias$ 。经过激活层后可得到目标输出特征图。

在计算图中,如果当前算子的后继算子仍为量化算子,那么当前算子的输出特征图经过量化后成为后继算子的输入特征图。为了提升硬件执行的效率,SAF-CNN 加速器在写回输出特征图时,预先进行

量化操作处理,供后继算子直接计算。在量化输出特征图时,使用下一层的 is 作为当前层输出缩放因子,这里以 os 表示,故 SAF-CNN 的具体量化处理为

$$output'' = clip(round(output'/os)), \quad (2)$$

其中 $output'$ 是量化后的输出特征图,函数 $clip$ 和 $round$ 分别代表截断和取整操作。考虑到 os 可以与 is , ws , $bias$ 进行预先处理。因此,结合式(1)(2)可以得到

$$output'' = clip(round(ws' \times output' + bias')), \quad (3)$$

其中 ws' 和 $bias'$ 分别由 ws 和 $bias$ 经过预处理得到。加速器将量化后的 $output''$ 写回到片外存储器中,供后继节点使用。通过这种方式,片上存储器和片外存储器之间传输的数据都是量化后的低位宽定点数据,可有效地降低带宽需求。

3.3 模型计算图重构

经过算子融合后,目标加速器的片上存储和片外存储之间传输的数据都是低位宽定点数据,只要符合输入输出数据类型的卷积算子都能够被调度到硬件 FPGA 平台进行执行,并且连续的卷积算子可以构成子图,交由硬件 SDK 进行执行。然而,由于数据类型的不匹配,CNN 模型的计算图中普遍存在量化后的卷积算子无法调度到 FPGA 平台进行加速的情况,主要原因是量化的卷积算子的后继节点是无法进行量化的算子,例如 slice 算子、transpose 算子等。针对这种情况,SAF-CNN 对模型计算图进行重构,在量化算子后加入量化节点和数据类型转换算子,来辅助完成量化算子调度到 FPGA 平台进行加速。以下分为 2 种情形来说明。

1) 单路径情形。CNN 模型计算图中存在的单路径情形如图 7(a)所示,量化算子 q_i 的输入数据类型是量化后的定点数,后继算子是非量化算子。因此,

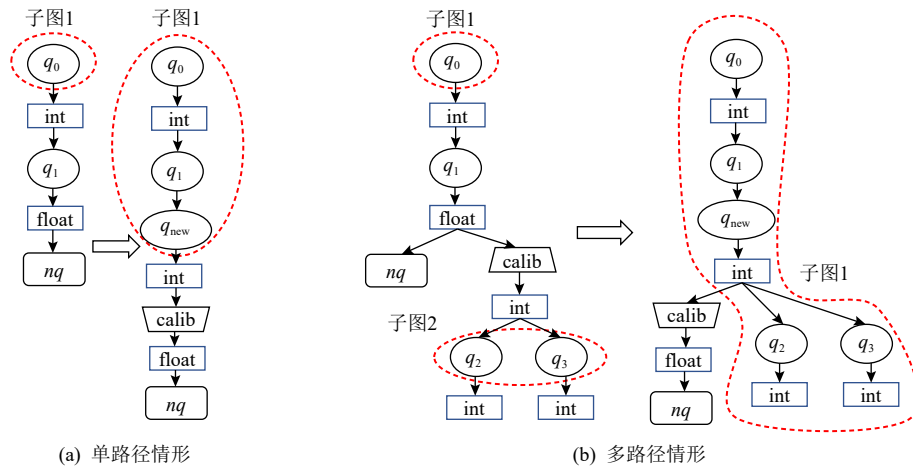


Fig. 7 Model reconstruction mechanism of inserting quantization node

图7 插入量化节点模型重构机制

q_1 的输出数据类型是浮点数据类型, 与加速器所期望的定点数输出数据类型是不匹配的. 在这种情形中, q_1 算子不能被调度到 FPGA 加速器平台执行. 通过插入 q_{new} 量化节点, 可对 q_1 算子的输出特征图进行量化, 并传递量化参数到 q_1 算子. 由此, 可实现 q_1 算子的输入数据类型与输出数据类型皆符合硬件加速器的设计要求. 需要指出的是, q_{new} 节点并不引入额外的计算量, 其实现时已被融合到加速器设计中. 此外, 利用数据类型转换算子可将低位宽的定点数据恢复为浮点数类型, 供后继的非量化算子使用.

2) 多路径情形. CNN 模型中存在的多路径情形如图 7(b) 所示, q_1 算子具有多个后继算子, 同时包含量化算子 q_2 , q_3 和非量化算子 nq . 与单路径情形相同, nq 后继算子要求 q_1 节点的输出特征图是浮点数类型, 导致 q_1 算子无法调度到硬件 FPGA 上执行. 此时, CNN 模型的计算图被分割为多个计算子图, CPU 和 FPGA 执行环境的切换将增大推理的延迟. 通过插入量化节点可以统一数据类型, 并将多个计算子图合并为更大规模的计算子图, 从而使 FPGA 平台执行更多的算子, 进而减少执行环境的切换次数.

3.4 硬件子图融合

本文将硬件子图定义为具有相同数据依赖的硬件算子集合, 即在 CNN 计算图优化过程中, 多个具备数据依赖关系的算子可被封装成为 1 个子图, 该子图可作为这些算子的集合直接交由硬件 FPGA 执行. 硬件子图在 CNN 计算图的优化过程中产生, 并在推理过程中执行. 因此, 各个算子的执行顺序以及硬件需要的配置参数应在子图产生过程中确定, 从而避免在运行时引入额外的处理时间. 针对 CNN 模型的计算图, 本文采用深度优先搜索方式确定算子的

执行顺序. 考虑到 CNN 模型拓扑结构的多样性, 硬件子图可能具有多个输入和多个输出. 由于输入数据来自于硬件子图的外部节点, 输出数据供外部节点使用, 因此在硬件子图的输入节点前面或输出节点后面插入数据重组节点, 该节点对外部存储器中的数据进行重组.

如图 6 所示, 硬件 SDK 采用指令形式首先对加速器平台进行配置, 然后启动加速器. 该指令包含卷积层的参数配置, 如输入特征图的通道数、行数和列数以及卷积操作的配置参数. 此外, 为了减少硬件资源占用, 其他相关参数可预先进行配置, 如分块的个数、行数等. 针对输入特征图、权重和输出特征图的数据, 采用基地址加偏置的寻址方式进行访问, 该偏置参数也被预置在指令中.

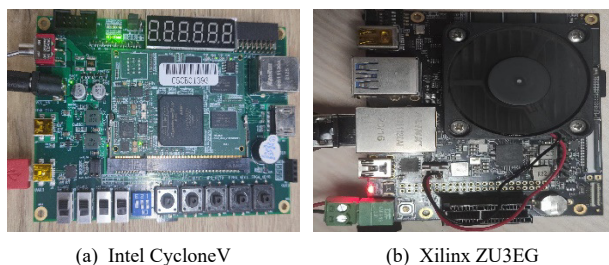
4 实验评估

本节通过实验评估 SAF-CNN 稀疏化加速器的设计, 并将 SAF-CNN 推理框架实现方法作为组件嵌入到深度学习推理框架 Paddle Lite2.11, 验证 SAF-CNN 加速器与框架联合优化的优势. 评估实验数据集分别采用 ImageNet^[48] 和 VOC2012 数据集^[49], 并部署典型 CNN 模型 VGG16^[2], ResNet18^[50], MobileNetV1^[19] 到资源极端受限的边缘嵌入式设备 CycloneV 和 ZU3EG 开发板, 探究 SAF-CNN 带来的性能收益.

4.1 实验设置

1) 硬件配置. 实验采用 Xilinx ZU3EG SoC FPGA 开发板和 Intel CycloneV FPGA 开发板作为硬件加速器验证平台, 如图 8 所示, FPGA 的运行频率分别设定为 333 MHz 和 150 MHz. 此外, 配备工作站运行

Vitis 软件与 Quartus 软件, 搭载 Intel Xeon Silver 4210 CPU@ 2.20 GHz 和 64GB DDR4 内存. 配备 1 块 GeForce RTX2080Ti GPU 显卡, 用于模型训练.



(a) Intel CycloneV

(b) Xilinx ZU3EG

Fig. 8 Hardware accelerator verification platform

图 8 硬件加速器验证平台

2) 软件配置. 对于 Xilinx FPGA 平台, 实验采用集成开发环境 Vitis2021.1 进行 FPGA 加速器设计. 通过编程语言 C++ 设计各模块, 并利用 Vitis HLS 工具来进行高层次综合. 对于 Intel FPGA 平台, 实验采用集成开发环境 Quartus17.1 进行 FPGA 加速器设计. 通过编程语言 C++ 设计各模块, 并利用 Quartus HLS 工具来进行高层次综合; 此外, 为进一步验证 SAF-CNN 的性能, 实验也采用寄存器传输级 (register-transfer level, RTL) 语言 Verilog-2001, 在 Intel FPGA 平台上实现并部署.

3) 评估指标. 面向嵌入式 FPGA 的 CNN 稀疏化加速框架 SAF-CNN 的实验评估选取 4 个定量指标来测量数据并分析性能收益.

① CNN 模型各层处理时延. 该指标能够反映出加速器针对不同配置的卷积层所能实现的计算性能, 并对比稀疏 CNN 加速器和普通 CNN 加速器的处理时延, 反映出 SAF-CNN 所设计的结构化剪枝方法带来的性能增益.

② 模型精度. 为验证 SAF-CNN 稀疏化剪枝算法对模型精度的影响, 实验以 VGG16, ResNet18, MobileNetV1 和目标检测模型 SSD_MobileNetV1 为例, 探究不同的稀疏率条件下模型精度的变化.

③ 模型端到端推理时延. 为验证 SAF-CNN 稀疏化加速器与推理框架实现的联合优化方法, 实验以包含深度可分离卷积的 SSD_MobileNetV1 模型为例, 以此验证 SAF-CNN 加速框架的推理性能.

④ 模型计算图重构性能增益. 为验证 SAF-CNN 对重构前模型和重构后模型的推理性能影响, 实验对比 2 个模型在相同平台的推理延迟, 并细粒度分析推理框架与加速器各个模块的执行时间, 呈现 SAF-CNN 模型计算图重构的优势.

4.2 稀疏化卷积处理性能评估

本节对 SAF-CNN 稀疏化剪枝算法的有效性进行评估, 利用结构化剪枝算法在 2 类不同平台上构建稀疏化加速器, 并与非剪枝算法的普通加速器及其他加速器方案进行性能对比. 稀疏化加速器 SAF-CNN_Sparse 设置 75% 的剪枝率, 计算引擎分别采用 64, 128, 256 个乘累加 (MAC) 单元进行验证. 输入通道并行因子和输出通道并行因子配置如表 1 所示. 其中, 输出通道并行因子和输入通道并行因子采用的 2 种配置分别是 16 和 32, 经过剪枝后输入通道并行因子被压缩为 4 和 8. 所对比的非剪枝算法的普通稠密加速器 SAF-CNN_Dense 方案则采用 64 个 MAC 单元, 输入和输出通道并行因子皆配置为 8. 此外, SAF-CNN 只对逐点卷积进行结构化剪枝, 逐通道卷积并没有引入稀疏性. 因为逐通道卷积的输入通道个数为 1, 不需要在输入通道维度进行剪枝.

Table 1 Accelerator MACs Unit Configuration

表 1 加速器乘累加单元配置

加速器	输出通道并行因子	输入通道并行因子	剪枝后输入通道并行因子	MACs 个数
SAF-CNN_Dense	8	8		64
	16	16	4	64
SAF-CNN_Sparse	32	16	4	128
	32	32	8	256

SAF-CNN 在 ImageNet 数据集上进行了图像分类实验, VGG16, ResNet18, MobileNetV1 这 3 个模型在 ZU3EG 平台上的分层处理时延如图 9 所示. 其中, MobileNetV1 的第 2~14 层是深度可分离卷积层, 每个深度可分离卷积层包含 1 个逐通道卷积和 1 个逐点卷积. VGG16 模型卷积层的通道数和特征图的尺寸大于 ResNet18 和 MobileNetV1 模型, 因此, VGG16 模型各层消耗的时间高于 ResNet18 和 MobileNetV1 模型. 图 9 中, 与非剪枝算法的普通稠密加速器相比, 稀疏加速器的处理速度更快. 具体来讲, 在 VGG16 和 ResNet18 模型中, 64 个 MACs 的稀疏化加速器与稠密加速器相比, 可分别获得 2.91 倍和 2.86 倍的计算性能提升, 十分接近理论上的 3 倍性能提升. 实际性能提升与理论性能提升的差距在于, 加速器对卷积层的处理时延不仅包含计算时间而且包含数据拷贝时间, 其中, 稀疏化加速器中输入特征图和输出特征图的数据拷贝时间和稠密加速器相同. 尽管 SAF-CNN 稀疏化加速器设计了计算与数据传输流水处理的方案, 但是在流水线第 1 个分块和最后 1 个分块的

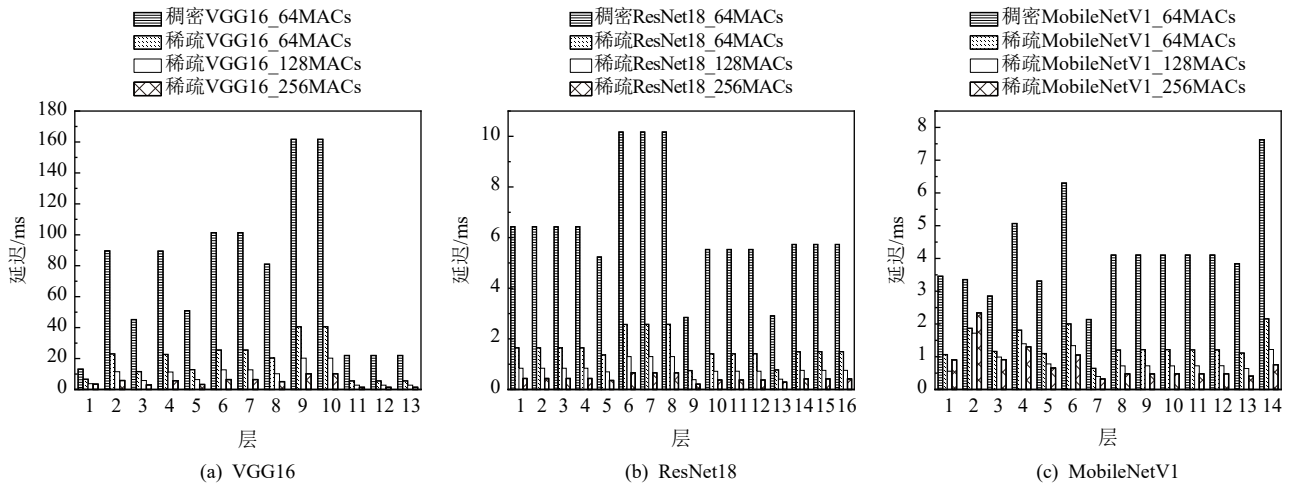


Fig. 9 Layer-wise latency of VGG16, ResNet18 and MobileNetV1 on ZU3EG

图9 ZU3EG 平台上 VGG16, ResNet18, MobileNetV1 的各层延迟

处理过程中, 加速器仍然有额外的数据传输时间. 因此, 稀疏化加速器的计算性能相比理论性能略有下降. 在 MobileNetV1 中, 计算引擎针对逐通道卷积的处理包含对填充数据的额外处理, 因此其计算性能略低于 VGG16 和 ResNet18. 因为逐通道卷积并没有引入稀疏性, 稀疏化加速器对逐通道卷积的计算性能和稠密加速器相同. 因此, 稀疏化加速器和稠密加速器相比有 2.1 倍的加速比. 此外, 采用 128 个 MACs 和 256 个 MACs 可进一步提升稀疏化加速器的性能. 与采用 64 个 MACs 的稀疏化加速器相比, 128 个 MACs 和 256 个 MACs 的加速器分别能够达到 1 倍和 2.86 倍的性能提升, 且分别实现 254.9 GOPS 和 494.3 GOPS 的计算性能. 在 CycloneV 平台上, 基于 HLS 和 RTL 所实现的加速器皆采用 64 个 MACs 进行乘累加操作, VGG16, ResNet18, MobileNetV1 这 3 个稀疏化模型的分层处理时延如图 10 所示. 与 HLS 加速器相比, RTL 加速器设计通过更细粒度的控制, 能够实现较低的处理延迟, 计算性能提升近 1 倍. 由

各个模型的计算性能可知, 稀疏化加速器能够高效处理普通卷积和深度可分离卷积.

SAF-CNN 稀疏化加速器与其他同是基于 Xilinx 的加速器的资源与性能对比如表 2 所示, 基于 CycloneV 的 SAF-CNN 加速器资源与性能如表 3 所示. 其中, 对比对象主要使用具有较大规模计算资源的 Xilinx FPGA 平台进行验证, 从而取得相对较高的计算性能. SAF-CNN 的加速器平台虽受限于资源条件, 但仍能在 CycloneV 平台分别取得 41.8 GOPS 和 76.3 GOPS 的计算性能, 在 ZU3EG 平台分别取得 128 GOPS, 254.9 GOPS, 494.3 GOPS 的计算性能. 通过对比每千个 LUT 的计算效率、单个 DSP 的计算效率及加速器功耗来评估加速器的计算效率, 如图 11 所示. 其中, ZU3EG_64 表示使用 64 个 MACs, 以此类推. 如图 11 (a) 所示, 和其他加速器相比, SAF-CNN 功耗代价更低, CycloneV 仅消耗 5 W, ZU3EG 消耗 7.5 W 左右. 此外, 在 ZU3EG 平台上 SAF-CNN 采用 3 种不同的 MACs 资源配置进行验证. 针对 LUT 效率, SAF-CNN 分别

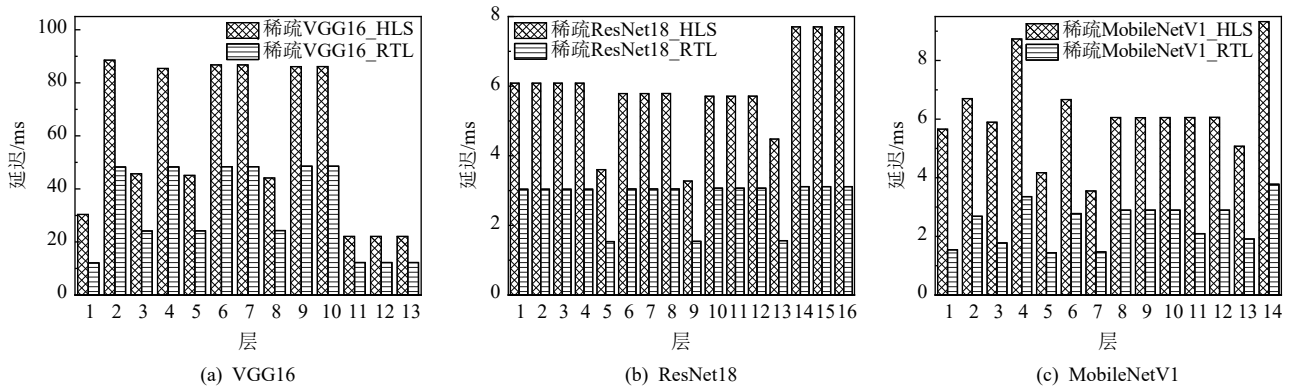


Fig. 10 Layer-wise latency of VGG16, ResNet18 and MobileNetV1 on CycloneV

图 10 CycloneV 平台上 VGG16, ResNet18, MobileNetV1 的各层延迟

Table 2 Computing Performance and Resource Utilization Comparison of ZU3EG Accelerators and Other Accelerators**表 2 ZU3EG 加速器与其他加速器计算性能与资源占用对比**

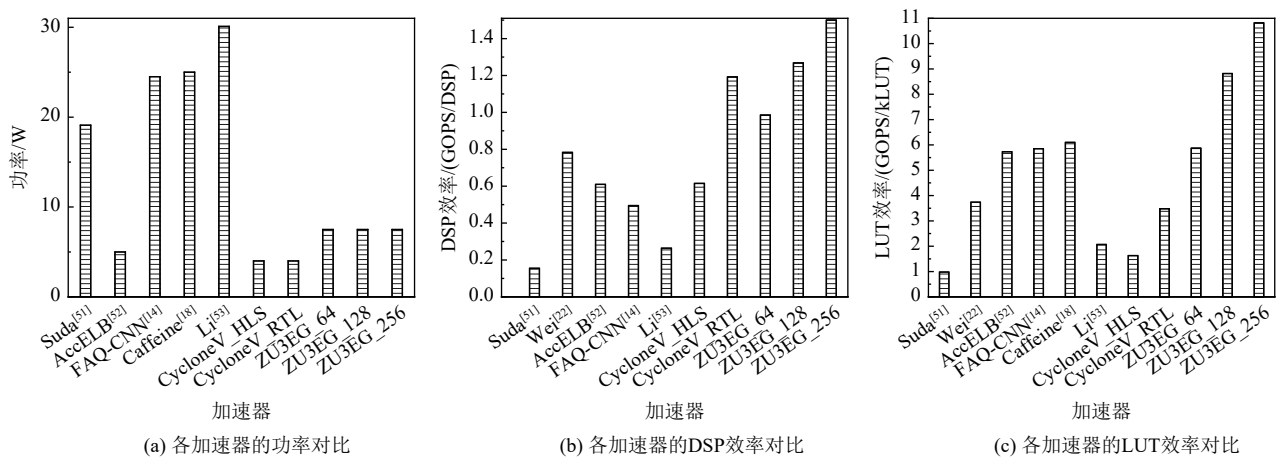
加速器	平台	频率/MHz	精度/b	kLUTs	DSP	BRAM	计算性能/GOPS
Suda ^[51]	Stratix-V GSD8		16	120	760		117.8
Wei ^[22]	Arria 10 GT1150	232	8	313	1 500	834	1 171.3
AccELB ^[52]	ZC706		8	86	808	303	493
FAQ-CNN ^[14]	ZU102	200	8	210	2 483	723	1 229
Caffeine ^[18]	KU060	200	8	200	116	784	1 200
Li ^[53]	Virtex-7 VC709	200	16	273	2 144	956	565.9
ZU3EG_64	ZU3EG	333	8	21.8	130	71	128
ZU3EG_128	ZU3EG	333	8	28.9	201	107	254.9
ZU3EG_256	ZU3EG	333	8	45.7	330	178	494.3

Table 3 Resource Utilization and Computing Performance of SAF-CNN Accelerator on CycloneV**表 3 SAF-CNN 加速器在 CycloneV 上的资源占用与计算性能**

编程语言	频率/MHz	kALMs	DSP	存储/Mb	计算性能/GOPS
RTL	150	21.9	64	2.5	76.3
HLS	150	25.7	68	4.1	41.8

实现 5.87 GOPS/kLUT, 8.82 GOPS/kLUT, 10.82 GOPS/kLUT 的计算性能. 针对 DSP 效率, SAF-CNN 分别实现 0.99 GOPS/DSP, 1.27 GOPS/DSP, 1.5 GOPS/DSP 的

计算性能, 远高于其他加速器. SAF-CNN 基于 CycloneV 平台的加速器采用的是 28 nm 的工艺, 却仍能运行在 150 MHz 的时钟频率. 在基于 HLS 的加速器设计上实现 1.63 GOPS/kLUT 和 0.62 GOPS/DSP 的计算性能, 明显高于资源丰富的 Suda 加速器^[51]和 Li 加速器^[53]. SAF-CNN 在基于 CycloneV 的 RTL 加速器设计上实现了高达 3.48 GOPS/kLUT 和 1.19 GOPS/DSP 的计算性能. 和 HLS 加速器相比, RTL 加速器进行更细粒度的任务调度, 通过数据加载模块和计算模块并行执行来充分隐藏数据传输代价, 从而实现更高的计算性能.

**Fig. 11 Power and computing efficiency comparison of accelerators****图 11 加速器功耗和计算效率对比**

4.3 模型精度评估

本节探究结构化剪枝方法对模型精度的影响. 在结构化剪枝算法中, SAF-CNN 采用 2 阶段的剪枝方法进行剪枝: 第 1 阶段进行非结构化剪枝以引入稀疏性; 第 2 阶段使用结构化剪枝. 同样, 输入通道并行因子 tn 和输出通道并行因子 tm 都配置为 16. 此外,

VGG16 和 ResNet18 模型的稀疏率从 10% 增加到 80%. SSD_MobileNetV1 模型的稀疏率从 10% 增加到 75%. 考虑到参数的冗余性, MobileNetV1 骨干网络的逐点卷积和 SSD 头部网络的普通卷积层都进行剪枝操作, 逐通道卷积不进行剪枝操作. 实验中加速器所设置的可处理输入通道个数为 16, 在稀疏率为 75%

的情况下, 经过 SAF-CNN 稀疏剪枝方法得到的输入通道个数减少到 4. 当块内输入通道大于 4, 则需裁剪为 4; 对于块内输入通道数等于或者小于 4 的输入特征图, 可无需裁减而直接输入到加速器进行计算. 由于 3 个模型处理的 RGB 图像数据的输入通道个数为 3, 此时可将该图像数据直接输入到 SAF-CNN 加速器直接计算, 而无需对模型的第 1 层进行稀疏剪枝, 进而不会对第 1 个卷积层引入稀疏性, 由此保证模型精度, 因为模型第 1 层的精度往往会直接影响整个模型推理精度.

在量化模型的基础上, 不同稀疏率下的模型准确率损失如图 12 所示. 在 Cifar100 数据集上, VGG16 模型的推理准确率是 71.86%, ResNet18 模型的推理准确率是 76.45%. 从图 12 可知, 在 VGG16 模型上采用 80% 的稀疏率时, 第 2 阶段剪枝算法可实现 0.08% 的准确率提升. 当采用小于 80% 的稀疏率时, ResNet18 模型的推理准确率损失小于 1%, 实现了理想的剪枝结果. 为验证剪枝方法在大规模数据集上的有效性, 实验验证 ResNet18 模型在 ImageNet 数据集的剪枝效果, 如图 12(c) 所示, 其中 ResNet18 模型在 ImageNet 数据集的推理准确率是 70.59%. 当稀疏率小于 60% 时, 剪枝算法会带来准确率提升, 当稀疏率设定为 80%

时, 剪枝算法仅导致 0.83% 的准确率损失. SSD_MobileNetV1 模型在 VOC2012 数据集上的推理平均准确率 mAP 为 73.8%, 经过 QAT 8b 量化算法处理后, mAP 为 72.59%. 其中, 当采用 50% 的稀疏率时, 模型的 mAP 损失小于 1%, 表明结构化算法的有效性. 当设定为 75% 的稀疏率时, SAF-CNN 结构化剪枝算法引入了 3.59% 的 mAP 损失. 针对该损失, SSD_MobileNetV1 模型采用深度可分离卷积降低模型冗余度, 可通过知识蒸馏技术更好地恢复精度.

4.4 模型端到端推理性能评估

本节实验通过部署 SSD_MobileNetV1 模型并评测其推理时延, 来揭示 SAF-CNN 加速器与框架实现联合优化所带来的性能增益. 实验采用 8b 的量化方法以及 75% 的稀疏率, 并在 Intel FPGA 和 Xilinx FPGA 这 2 种平台分别部署了 SAF-CNN 稀疏化加速框架. 此外, 将 SAF-CNN 与针对深度可分离卷积定制的 DPU 加速器^[20]进行对比.

SAF-CNN 加速器和 DPU 加速器的运行频率, 以及硬件资源使用量如表 3 和表 4 所示. 当联合深度学习框架时, SAF-CNN 在 ZU3EG 平台上实现数据重组功能, 会导致资源使用量略有提升. CycloneV 运行时钟频率为 150 MHz, DPU 和 ZU3EG 则基于 16 nm 工

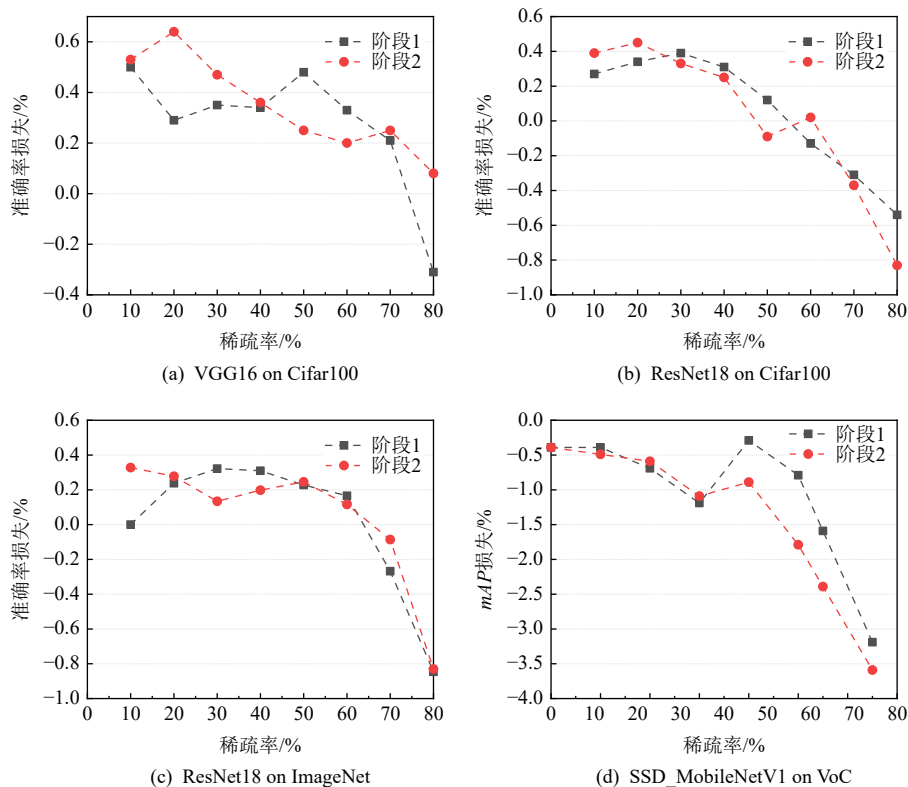


Fig. 12 Accuracy loss of CNN models with different sparsity ratios

图 12 不同稀疏率下 CNN 模型的精度损失

Table 4 Hardware Configuration Comparison of Accelerators**表 4 加速器硬件配置对比**

平台	硬件型号	频率/MHz	LUT	BRAM	DSP
DPU_S	ZU2EG	430	31 198	145	212
DPU_L	ZU9EG	333	161 944	711	2 070
ZU3EG_64	ZU3EG	333	26 575	84	137
ZU3EG_128	ZU3EG	333	33 496	120	208
ZU3EG_256	ZU3EG	333	50 506	191	337

Table 5 Inference Performance Comparison of SSD_MobileNetV1**表 5 SSD_MobileNetV1 推理性能对比**

平台	编程语言	硬件配置	帧率/fps	DSP 效率/ (fps/DSP)
CPU0	C++	ARMV7 * 2	2.2	
CPU1	C++	ARMV8 * 4	8.3	
DPU_S		ZU2EG	31	0.146
DPU_L		ZU9EG	124.3	0.06
CycloneV_64	HLS	ARMV7+CycloneV	5.3	0.078
CycloneV_64	RTL	ARMV7+CycloneV	10	0.156
ZU3EG_64	HLS	ARMV8+ZU3EG	20.2	0.147
ZU3EG_128	HLS	ARMV8+ZU3EG	25.6	0.123
ZU3EG_256	HLS	ARMV8+ZU3EG	26.5	0.079

艺,可运行在更高的时钟频率.和DPU相比,SAF-CNN加速器可使用更少的硬件资源,尤其是乘法器DSP,这主要得益于SAF-CNN设计了结构化剪枝算法.表5展示了各个平台上的SSD_MobileNetV1加速器推理性能对比,CPU0和CPU1分别是采用ARMV7架构的2核处理器和ARMV8架构的4核处理器.在加速器与框架结合的方案中,所有的卷积层都调度到FPGA执行,其他算子仍然在CPU上执行.在CycloneV平台上采用HLS设计时,SAF-CNN稀疏化异构加速器和CPU相比能够获得1.4倍的性能提升,采用RTL实现的加速器则能够获得3.5倍的性能提

升.在ZU3EG平台上采用HLS的SAF-CNN异构加速器,使用64 MACs,128 MACs,256 MACs分别能够获得1.8倍、2.1倍和2.2倍的性能提升.DPU凭借自身高运行频率和丰富的资源量分别达到了31 fps和124.3 fps的推理性能.其中DPU_L不仅对深度可分离卷积算子实现加速,而且包含了SSD_MobileNetV1中其他大量的轻量级算子,从而达到了124.3 fps的推理性能.但为了公平对比,本文比较了单位DSP上的SAF-CNN和DPU的处理性能.SAF-CNN在CycloneV平台上,采用RTL实现0.156 fps/DSP的计算性能,在ZU3EG平台上采用64个MACs能够实现0.147 fps/DSP的计算性能,高于DPU_S的0.146 fps/DSP的计算性能.说明SAF-CNN的设计能够在单位硬件资源上实现较高的计算能力,该方法如应用在资源丰富的FPGA平台将获得更高性能.

4.5 模型计算图重构性能增益

本节对SAF-CNN中模型计算图重构的有效性进行评估,在ZU3EG平台分别使用64个MACs、128个MACs和256个MACs设计加速器,并对SSD_MobileNetV1模型重构前和重构后的推理性能进行对比.在SSD_MobileNetV1中,需要进行模型重构的计算图结构主要分布在MobileNetV1骨干网络和SSD头部网络的连接部分.

经过实测,SAF-CNN所实现的SSD_MobileNetV1模型,其推理时延的各部分组成与占比数据如表6所示.其中ZU3EG_64表示使用64个MACs,以此类推.表6中FPGA硬件子图运行时间包含了调度到FPGA平台上的所有卷积层的运行时间,框架处理时延包含轻量级的算子执行时间和未被调度到FPGA的数据类型不匹配的卷积运行时间.模型计算图重构前,部分数据类型不匹配的卷积算子被调度到CPU执行,导致所有的卷积层被分成6个子图.此外,CPU端执行18个卷积层,导致框架处理时延显著提升.当经过模型计算图重构后,所有的卷积层都是互

Table 6 Inference Latency Components of SSD_MobileNetV1**表 6 SSD_MobileNetV1 推理延迟组成**

推理结果	ZU3EG_64		ZU3EG_128		ZU3EG_256	
	计算图重构前	计算图重构后	计算图重构前	计算图重构后	计算图重构前	计算图重构后
子图数量	6	1	6	1	6	1
CPU 端卷积层数量	18	0	18	0	18	0
数据重排时延/ms	5.1	3.1	4.7	3.1	4.9	3.1
FPGA 子图运行时延/ms	27.1	32.1	17.1	22	15.1	20
框架处理时延/ms	78.9	14.4	81	14.5	79.5	14.7
总时延/ms	111.1	49.6	102.8	39.6	99.5	37.8

连的并且都可被调度到 FPGA 进行加速, 因此只有 1 个子图. 由于子图数目的减少, 数据重排的次数也会降低, 促使数据重排的时延由 5 ms 左右降低到 3 ms 左右. 卷积层都被调度到 FPGA 平台进行加速, 使得 FPGA 的运行时间分别从 27.1 ms 增加到 32.1 ms、从 17.1 ms 增加到 22 ms 和从 15.1 ms 增加 20 ms. CPU 端只负责执行轻量级的算子, 从而使框架处理时延分别由 78.9 ms 降低为 14.4 ms、81 ms 降低为 14.5 ms 和 79.5 ms 降低为 14.7 ms, 充分发挥加速器的计算优势. 当增加 MAC 乘累加单元数量时, SSD_MobileNetV1 的推理总时延从 49.6 ms 降低到 37.8 ms, 实现了更高的推理速度. 然而, 以下 3 方面制约了 SAF-CNN 在该平台进一步提升性能: 1) 该平台资源有限, 相比于 DPU_L, 只能在有限条件下增加计算引擎的并行能力和片上缓存; 2) 通过增加输入通道并行因子和输出通道并行因子来增加 MAC 使用个数, 会增加逐通道卷积的无效计算量, 从而导致深度可分离卷积的计算性能没有明显提升; 3) SSD_MobileNetV1 模型中包含的其他算子被调度到 CPU 上进行执行, 该部分算子执行过程约占用 14.5 ms, 使得整个模型的推理性能受到限制. 即使在资源和 CPU 性能受限的条件下, SAF-CNN 经过模型计算图重构优化后, 推理总时延分别是 49.6 ms, 39.6 ms 和 37.8 ms, 仍能实现重构前的 2.24 倍、2.6 倍和 2.63 倍. 由此证明, SAF-CNN 的模型计算图重构方法可以有效提升 SAF-CNN 加速器与框架结合的推理性能.

5 总 结

本文提出了一种面向嵌入式 FPGA 的稀疏化神经网络加速框架 SAF-CNN, 分别从加速器设计与推理框架实现方面进行设计与联合优化. 通过设计稀疏化架构、块内稀疏剪枝方法、资源复用机制、数据流设计、数据存储通信等技术实现了 SAF-CNN 稀疏化加速器. 然后, SAF-CNN 通过设计 CNN 量化模型计算图重构等方法, 对推理框架与模型目标加速器进行联合优化, 从而实现快速部署 CNN 模型到资源受限的嵌入式 FPGA 平台. 实验结果表明, 即使在资源受限的低端 FPGA 平台上, 如 CycloneV 和 ZU3EG, SAF-CNN 仍可实现高达 76.3 GOPS 和 494.3 GOPS 的计算性能. 在目标检测模型推理中, SAF-CNN 加速器与多核 CPU 相比可达到 3.5 倍和 2.2 倍的推理速度提升, 实现 26.5 fps 的优越性能, 且与同类方案相比, SAF-CNN 的 DSP 效率具有明显优势.

作者贡献声明: 谢坤鹏提出算法实现思路, 完成实验并撰写论文; 仪德智负责实验数据整理和分析; 刘义情参与协助完成部分实验; 刘航调研方案和数据分析; 赫鑫宇负责论文校对和图表修正; 龚成参与论文校对和实验数据审查; 卢冶提出算法思路、实验方案, 修改与审核全文.

参 考 文 献

- [1] Shafique M, Theodorides T, Reddy V J, et al. TinyML: Current progress, research challenges, and future roadmap[C]//Proc of the 58th ACM/IEEE Design Automation Conf (DAC). Piscataway, NJ: IEEE, 2021: 1303–1306
- [2] Simonyan K, Zisserman A. Very deep convolutional networks for large-scale image recognition[J]. arXiv preprint, arXiv: 1409.1556, 2014
- [3] Ren S, He Kaiming, Girshick R, et al. Faster R-CNN: Towards real-time object detection with region proposal networks[J]. IEEE Transactions on Pattern Analysis & Machine Intelligence, 2017, 39(6): 1137–1149
- [4] He Kaiming, Gkioxari G, Dollár P, et al. Mask R-CNN[C]//Proc of the 16th Int Conf on Computer Vision. Piscataway, NJ: IEEE, 2017: 2961–2969
- [5] Li Guang, Wang Peisong, Liu Zejian, et al. Hardware acceleration of CNN with one-hot quantization of weights and activations [C]//Proc of the 23rd Design, Automation & Test in Europe Conf & Exhibition (DATE). Piscataway, NJ: IEEE, 2020: 971–974
- [6] Cong J, Xiao Bingjun. Minimizing computation in convolutional neural networks[C]//Proc of the 24th Int Conf on Artificial Neural Networks. Berlin: Springer, 2014: 281–290
- [7] Prost-Boucle A, Bourge A, Pétrot F, et al. Scalable high-performance architecture for convolutional ternary neural networks on FPGA[C/OL]//Proc of the 27th Int Conf on Field Programmable Logic and Applications (FPL). Piscataway, NJ: IEEE, 2017[2019-06-10]. <https://ieeexplore.ieee.org/document/8056850>
- [8] Mousoulitis P G, Petrou L P. CNN-Grinder: From algorithmic to high-level synthesis descriptions of CNNs for low-end-low-cost FPGA SoCs[J]. Microprocessors and Microsystems, 2020, 73: 102990
- [9] Chen Guilin, Ma Sheng, Guo Yang. Survey on accelerating neural network with hardware[J]. Journal of Computer Research and Development, 2019, 56(2): 240–253 (in Chinese)
(陈桂林, 马胜, 郭阳. 硬件加速神经网络综述[J]. 计算机研究与发展, 2019, 56(2): 240–253)
- [10] Mishra R, Gupta H P, Dutta T. A survey on deep neural network compression: Challenges, overview, and solutions[J]. arXiv preprint, arXiv: 2010.03954, 2020
- [11] Courbariaux M, Hubara I, Soudry D, et al. Binarized neural networks: Training deep neural networks with weights and activations constrained to+1 or-1[J]. arXiv preprint, arXiv: 1602.02830, 2016

- [12] Li Fengfu, Zhang Bo, Liu Bin. Ternary weight networks[J]. arXiv preprint, arXiv: 1605.04711, 2016
- [13] Gondimalla A, Chesnut N, Thottethodi M, et al. SparTen: A sparse tensor accelerator for convolutional neural networks[C]//Proc of the 52nd Annual IEEE/ACM Int Symp on Microarchitecture. New York: ACM, 2019: 151–165
- [14] Xie Kunpeng, Lu Ye, Jin Zongming, et al. FAQ-CNN: A flexible acceleration framework for quantized convolutional neural networks on embedded FPGAs[J]. *Journal of Computer Research and Development*, 2022, 59(7): 1409–1427 (in Chinese)
(谢坤鹏, 卢冶, 靳宗明, 等. FAQ-CNN: 面向量化卷积神经网络的嵌入式FPGA可扩展加速框架[J]. *计算机研究与发展*, 2022, 59(7): 1409–1427)
- [15] Chollet F. Xception: Deep learning with depthwise separable convolutions[C]//Proc of the 30th IEEE/CVF Conf on Computer Vision and Pattern Recognition. Piscataway, NJ: IEEE, 2017: 1251–1258
- [16] Abadi M, Barham P, Chen Jianmin, et al. TensorFlow: A system for large-scale machine learning[C]//Proc of the 12th USENIX Symp on Operating Systems Design and Implementation. Berkeley, CA: USENIX, 2016: 265–283
- [17] Paszke A, Gross S, Massa F, et al. PyTorch: An imperative style, high-performance deep learning library[C]//Proc of the 32nd Conf on Neural Information Processing Systems. New York: Curran Associates, 2019: 8024–8035
- [18] Zhang Chen, Sun Guangyu, Fang Zhenman, et al. Cafféine: Toward uniformed representation and acceleration for deep convolutional neural networks[J]. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2018, 38(11): 2072–2085
- [19] Howard A G, Zhu Menglong, Chen Bo, et al. MobileNets: Efficient convolutional neural networks for mobile vision applications[J]. arXiv preprint, arXiv: 1704.04861, 2017
- [20] Wu Di, Zhang Yu, Jia Xijie, et al. A high-performance CNN processor based on FPGA for MobileNets[C]//Proc of the 29th Int Conf on Field Programmable Logic and Applications (FPL). Piscataway, NJ: IEEE, 2019: 136–143
- [21] Ding Wei, Huang Zeyu, Huang Zunkai, et al. Designing efficient accelerator of depthwise separable convolutional neural network on FPGA[J]. *Journal of Systems Architecture*, 2019, 97: 278–286
- [22] Wei Xuechao, Yu C H, Zhang Peng, et al. Automated systolic array architecture synthesis for high throughput CNN inference on FPGAs[C/OL]//Proc of the 54th Annual Design Automation Conf. New York: ACM, 2017[2020-05-08]. <https://ieeexplore.ieee.org/document/8060313>
- [23] Cong J, Wei Peng, Yu C H, et al. Automated accelerator generation and optimization with composable, parallel and pipeline architecture[C/OL]//Proc of the 55th ACM/ESDA/IEEE Design Automation Conf (DAC). Piscataway, NJ: IEEE, 2018[2021-01-03]. <https://ieeexplore.ieee.org/document/8465940>
- [24] Véstias M, Duarte R P, de Sousa J T, et al. A fast and scalable architecture to run convolutional neural networks in low density FPGAs[J/OL]. *Microprocessors and Microsystems*, 2020, 77 [2021-12-04]. <https://www.sciencedirect.com/science/article/pii/S0141933120303033>
- [25] Baidu. PaddlePaddle: An open source deep learning platform derived from industrial practice[EB/OL]. [2020-08-10]. <https://www.paddlepaddle.org.cn/> (in Chinese)
(百度. 飞桨: 源于产业实践的开源深度学习平台 [EB/OL]. [2020-08-10]. <https://www.paddlepaddle.org.cn/>)
- [26] Zhou Shuchang, Wu Yuxin, Ni Zekun, et al. Dorefa-net: Training low bitwidth convolutional neural networks with low bitwidth gradients[J]. arXiv preprint, arXiv: 1606.06160, 2016
- [27] Miyashita D, Lee E H, Murmann B. Convolutional neural networks using logarithmic data representation[J]. arXiv preprint, arXiv: 1603.01025, 2016
- [28] Han Song, Mao Huizi, Dally W J. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding[J]. arXiv preprint, arXiv: 1510.00149, 2015
- [29] Liu Zhuang, Li Jianguo, Shen Zhiqiang, et al. Learning efficient convolutional networks through network slimming[C]//Proc of the 16th Int Conf on Computer Vision. Piscataway, NJ: IEEE, 2017: 2736–2744
- [30] Ma Xiaolong, Lin Sheng, Ye Shaokai, et al. Non-structured DNN weight pruning—Is it beneficial in any platform?[J/OL]. *IEEE Transactions on Neural Networks and Learning Systems*, 2021[2022-05-01]. <https://ieeexplore.ieee.org/abstract/document/9381660>
- [31] Song Linghao, Chi Yuze, Guo Licheng, et al. Serpens: A high bandwidth memory based accelerator for general-purpose sparse matrix-vector multiplication[C]//Proc of the 59th ACM/IEEE Design Automation Conf (DAC). Piscataway, NJ: IEEE, 2022: 211–216
- [32] Li Shiyu, Hanson E, Qian Xuehai, et al. ESCALATE: Boosting the efficiency of sparse CNN accelerator with kernel decomposition[C]//Proc of the 54th Annual IEEE/ACM Int Symp on Microarchitecture. New York: ACM, 2021: 992–1004
- [33] Li Hao, Kadav A, Durdanovic I, et al. Pruning filters for efficient convnets[J]. arXiv preprint, arXiv: 1608.08710, 2016
- [34] Tan Zhanhong, Song Jiebo, Ma Xiaolong, et al. PCNN: Pattern-based fine-grained regular pruning towards optimizing CNN accelerators[C/OL]//Proc of the 57th ACM/IEEE Design Automation Conf (DAC). Piscataway, NJ: IEEE, 2020[2021-10-03]. <https://dl.acm.org/doi/10.5555/3437539.3437730>
- [35] Mao Huizi, Han Song, Pool J, et al. Exploring the granularity of sparsity in convolutional neural networks[C]//Proc of the 30th IEEE/CVF Conf on Computer Vision and Pattern Recognition Workshops. Los Alamitos, CA: IEEE Computer Society, 2017: 1927–1934

- [36] Zhang Chen, Li Peng, Sun Guangyu, et al. Optimizing FPGA-based accelerator design for deep convolutional neural networks[C]//Proc of the 23rd ACM/SIGDA Int Symp on Field-Programmable Gate Arrays. New York: ACM, 2015: 161–170
- [37] Lu Ye, Chen Yao, Li Tao, et al. Convolutional neural network construction method for embedded FPGAs oriented edge computing[J]. *Journal of Computer Research and Development*, 2018, 55(3): 551–562 (in Chinese)
(卢冶, 陈瑶, 李涛, 等. 面向边缘计算的嵌入式FPGA卷积神经网络构建方法[J]. *计算机研究与发展*, 2018, 55(3): 551–562)
- [38] Lu Liqiang, Liang Yun, Xiao Qingcheng, et al. Evaluating fast algorithms for convolutional neural networks on FPGAs[C]//Proc of the 25th Annual Int Symp on Field-Programmable Custom Computing Machines (FCCM). Piscataway, NJ: IEEE, 2017: 101–108
- [39] Wu Haoning, Huang C T. Data locality optimization of depthwise separable convolutions for CNN inference accelerators[C]//Proc of the 22nd Design, Automation & Test in Europe Conf & Exhibition (DATE). Piscataway, NJ: IEEE, 2019: 120–125
- [40] Zhang Xiaofan, Lu Haoming, Hao Cong, et al. SkyNet: A hardware-efficient method for object detection and tracking on embedded systems[J]. *Proceedings of Machine Learning and Systems*, 2020, 2: 216–229
- [41] Yan Shun, Liu Zhengyan, Wang Yun, et al. An FPGA-based MobileNet accelerator considering network structure characteristics[C]//Proc of the 31st Int Conf on Field Programmable Logic and Applications (FPL). Piscataway, NJ: IEEE, 2021: 17–23
- [42] Du Zidong, Fasthuber R, Chen Tianshi, et al. ShiDianNao: Shifting vision processing closer to the sensor[C]//Proc of the 42nd Annual Int Symp on Computer Architecture. New York: ACM, 2015: 92–104
- [43] Zhang Zhichao, Mahmud M A P, Kouzani A Z. FitNN: A low-resource FPGA-based CNN accelerator for drones[J/OL]. *IEEE Internet of Things Journal*, 2022[2022-08-01]. <https://ieeexplore.ieee.org/abstract/document/9785605>
- [44] Chen Tianqi, Moreau T, Jiang Ziheng, et al. TVM: An automated end-to-end optimizing compiler for deep learning[C]//Proc of the 13th USENIX Symp on Operating Systems Design and Implementation. Berkeley, CA: USENIX Association, 2018: 578–594
- [45] Li Rengang, Kan Hongwei, Su Dongdong, et al. An optimal design method of Conv2d operator for TensorFlow based on FPGA accelerator[C/OL]//Proc of the 4th Int Conf on Computer Science and Application Engineering. New York: ACM, 2020[2022-08-03]. <https://dl.acm.org/doi/10.1145/3424978.3424987>
- [46] Nunez-Yanez J. Fused architecture for dense and sparse matrix processing in TensorFlow Lite[J/OL]. *IEEE Micro*, 2022[2022-08-15]. <https://ieeexplore.ieee.org/abstract/document/9851516>
- [47] Martone M, Filippone S, Tucci S, et al. Use of hybrid recursive csr/coo data structures in sparse matrix-vector multiplication[C]//Proc of the Int Multiconference on Computer Science and Information Technology. Piscataway, NJ: IEEE, 2010: 327–335
- [48] Russakovsky O, Deng Jia, Su Hao, et al. ImageNet large scale visual recognition challenge[J]. *International Journal of Computer Vision*, 2015, 115(3): 211–252
- [49] Everingham M, Eslami S M, Van Gool L, et al. The pascal visual object classes challenge: A retrospective[J]. *International Journal of Computer Vision*, 2015, 111(1): 98–136
- [50] He Kaiming, Zhang Xiangyu, Ren Shaoqing, et al. Deep residual learning for image recognition[C]//Proc of the 29th IEEE/CVF Conf on Computer Vision and Pattern Recognition. Piscataway, NJ: IEEE, 2016: 770–778
- [51] Suda N, Chandra V, Dasika G, et al. Throughput-optimized OpenCL-based FPGA accelerator for large-scale convolutional neural networks[C]//Proc of the 24th ACM/SIGDA Int Symp on Field-Programmable Gate Arrays. New York: ACM, 2016: 16–25
- [52] Wang Junsong, Lou Qiuwen, Zhang Xiaofan, et al. Design flow of accelerating hybrid extremely low bit-width neural network in embedded FPGA[C]//Proc of the 28th Int Conf on Field Programmable Logic and Applications (FPL). Piscataway, NJ: IEEE, 2018: 163–1636
- [53] Li Huimin, Fan Xitian, Jiao Li, et al. A high performance FPGA-based accelerator for large-scale convolutional neural networks[C/OL]//Proc of the 26th Int Conf on Field Programmable Logic and Applications (FPL). Piscataway, NJ: IEEE, 2016[2019-05-20]. <https://ieeexplore.ieee.org/document/7577308>



Xie Kunpeng, born in 1996. PhD candidate. Member of CCF. His main research interests include heterogeneous computing, machine learning, and embedded system.
谢坤鹏, 1996年生. 博士研究生, CCF会员. 主要研究方向为异构计算、机器学习和嵌入式系统。



Yi Dezhi, born in 1999. Master candidate. Member of CCF. His main research interests include deep learning and heterogeneous computing.
仪德智, 1999年生. 硕士研究生. CCF会员. 主要研究方向为深度学习和异构计算。

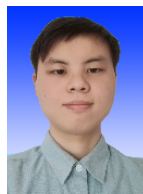


Liu Yiqing, born in 1994. Master candidate. His main research interests include DNN model automatic pruning, model transformation tool, and FPGA deployment implementation.
刘义情, 1994年生. 硕士研究生. 主要研究方向为DNN模型自动剪枝、模型转换工具和FPGA部署实现。



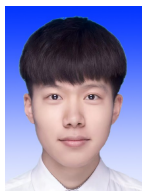
Liu Hang, born in 2000. Master candidate. His main research interests include machine learning and FPGA deployment implementation.

刘 航, 2000 年生. 硕士研究生. 主要研究方向为机器学习和 FPGA 部署实现.



Gong Cheng, born in 1993. PhD, lecturer. His main research interests include heterogeneous computing, machine learning, and Internet of things.

龚 成, 1993 年生. 博士, 讲师. 主要研究方向为异构计算、机器学习和物联网.



He Xinyu, born in 1996. PhD candidate. Member of CCF. His main research interests include computer vision, machine learning, and model compression and acceleration.

赫鑫宇, 1996 年生. 博士研究生. CCF 会员. 主要研究方向为计算机视觉、机器学习和模型压缩与加速.



Lu Ye, born in 1986. PhD, associate professor. Senior member of CCF. His main research interests include high performance embedded system, heterogeneous computing, and artificial intelligence.

卢 冶, 1986 年生. 博士, 副教授. CCF 高级会员. 主要研究方向为高性能嵌入式系统、异构计算和人工智能.